Entfernungsangaben in PfotenNetz

Bestandsaufnahme, Korrelationen und logische Bewertung

Stand: 8. November 2026 · Analyse und Umsetzungsstand · Grundlage: Mobile App, Web App, `packages/shared`, `packages/supabase`, Supabase-Migrationen

> **Kurzfazit:** Die App verwendet acht verschiedene Entfernungs- bzw. Radiuswerte, die teilweise denselben Namen tragen, aber unterschiedliche Dinge steuern. Drei Korrelationen waren fachlich falsch: Der Profil-Radius wurde als „Suchradius" bezeichnet, steuerte aber nicht die Helfersuche. Der Warnradius einer Gefahr und der Radar-Radius waren zwei verschiedene Dinge, sodass die Liste mehr zeigte, als das Detail lesen durfte. Und der Maximalradius einer Helfer-Verfügbarkeit blieb bei offenen Anfragen und Direktanfragen unberücksichtigt. Diese Punkte sind umgesetzt; der Stand steht in Abschnitt 5.

1. Inventar der Entfernungsangaben

#	Name in der App	Feld / Konstante	Werte & Default	Wirkung	Anzeige

1	„Nachbarschafts-Radius" (mobil/Web)	`profiles.kiez_radius_km`
		`shared/constants/app.ts:16–17`
		`supabase/profile/queries.ts:13–14`
		`001_profiles.sql:12`
			0,5 / 1,0 / 1,5 / 3,0 km
			Default 1,5
				Offene Anfragen (`helper_matches_open_booking`), Statistik und Startwert der Helfersuche	Profil (Chips + Info-Zeile)

2	Explore-Radius („Radius")	`RADII` transient
		`mobile/.../explore.tsx:31,223`
		`web/.../explore/page.tsx:32,232`
			0,5 / 1 / 1,5 / 3 / 5 / 10 km
			Start = Profil-Radius, Fallback 3
				`find_nearby_helpers(p_radius_km)`: Filter um den Live-GPS-Punkt; Backend erlaubt 0,5–50 (`20261025130000:66,87`)	Chips, Karte, „ca. ±X km"

3	„Maximalradius (km)" pro Zeitfenster	`helper_availabilities.max_distance_km`
		`012:11`, `025:26`
		Client-Check
		`availabilities/queries.ts:82–88`
			> 0 bis 50 km
			Default 5
				Filter in `find_nearby_helpers`: `a.max_distance_km >= distance_km` (`20261025130000:105`)	Helfer-Detail „bis X km" (`helper/[id].tsx:177`)

4	„Sicherheitszone" / Tracking-Radius	`bookings.tracking_radius_m` `20261024160000_widen_tracking_radius.sql`	100–10 000 m, Default 1 000 Optionen 1/2/4 km; frei 0,1–10 km	Geofence + Kreis auf der Live-Karte (`geofence.native.ts:41–56`, `live-tracking-map.tsx:16–23`)	Buchungsdetail, Live-Tracking
5	„Distanz" (gelaufen)	`tracking_sessions.total_distance_meters` `tracking/queries.ts:101–112`	Meter, keine Obergrenze	Kumulierte Haversine-Strecke (Intervall: 10 m / 10 s)	„x,xx km"
6	„Warnradius" einer Gefahr	`hazards.radius_km` `006_hazards.sql:33`	DB-Default 0,5; Check ≤ 10 App: 0,5 (nicht kritisch) / 1,5 (kritisch) Shared-Schema-Default 1,5	**RLS:** Gefahr, Sichtungen und Fotos nur lesbar, wenn der eigene Standort innerhalb dieses Radius liegt (`006:67–72`, `007:42–51`, `014:81–91`)	„Warnradius" im Detail, Report-Vorschau
7	Gefahren-Radar-Radius	`RADII` transient `hazard/radar.tsx:31,206` Web analog	0,5 / 1 / 1,5 / 3 km Default 1,5	RPC `get_active_hazards_in_radius_v2` (SECURITY DEFINER, umgeht RLS); erlaubt 0,1–10, Default 5 (`044`)	Chips, Karte
8	„Suchradius" vermisstes Tier	`missing_pets.search_radius_km` `008_missing_pets.sql:12`	DB-Default 3,0; Check ≤ 50 App: 3,0	**RLS:** Meldung nur lesbar, wenn der eigene Standort innerhalb des Melder-Radius liegt (`008:42–46`)	„Suchradius 3 km" am Eintrag

Zusätzlich existiert eine **Radius-Ersatzregel ohne km**: gleiche Postleitzahl (`helper_matches_open_booking`, OR-Verknüpfung, `20261021102000:227–231`).

2. Wer nutzt welchen Radius?

Aktion	Verwendeter Radius	Referenzpunkte
Helfer:innen in Explore finden	#2 (Liste) + #3 (Reisebereitschaft)	Live-GPS ↔ Helfer-Profilstandort
Entfernung im Helfer-Detail	**nicht** #2, sondern neu berechnet	**gespeicherter Profilstandort** ↔ Helfer
Offene Anfrage sehen / annehmen	#1 (+ PLZ)	Helfer-Profilstandort ↔ exakter Anfrageort
Anfrage direkt an Helfer senden	nur Zeitfenster/Typ, **kein Radius**	–
Gefahr im Radar sehen	#7	Live-GPS ↔ Gefahrenzentrum
Gefahr-Detail / Fotos / Sichtungen	#6	gespeicherter Profilstandort ↔ Gefahrenzentrum
Vermisstes Tier im Radar sehen	#8	gespeicherter Profilstandort ↔ letzter Sichtungsort
Betreuung während Tracking verlassen	#4	Tracking-Startpunkt
Gelaufene Strecke	#5	GPS-Punkte

3. Unlogische / fragile Korrelationen

Hinweis: Dieser Abschnitt dokumentiert den Ausgangsbefund vom 4. November 2026. Der Umsetzungsstand steht in Abschnitt 5.

A. „Suchradius" im Profil sucht nichts HOCH

- Web nennt `kiez_radius_km` „Suchradius" (`web/.../profile/page.tsx:366`), mobil „Nachbarschafts-Radius" (`profile.tsx:603`).
- Tatsächlich beeinflusst er nur, wer deine **offenen Anfragen** sieht. Die Helfersuche nutzt ausschließlich den transienten Explore-Radius (#2), der bei jedem Öffnen auf 3 km zurückfällt.
- Zwei verschieden benannte, aber gleich aussehende Radien: Nutzer erwarten zu Recht, dass der Profil-Radius die Suche bestimmt.

B. Offene Anfragen: Anbieter-Radius zählt, Helfer-Radius nicht HOCH

- Sichtbarkeit und Annahme hängen am **Seeker-Radius** (#1, max. 3 km) oder an gleicher PLZ.
- Der **Maximalradius des Helfers** (#3, bis 50 km) und das passende Zeitfenster werden weder bei der Sichtbarkeit noch bei `helper_accept_booking` geprüft.
- Folgen: Ein Helfer mit 30 km Reisebereitschaft bekommt nur Anfragen im 3-km-Kiez; ein Helfer mit 1 km Reisebereitschaft sieht Anfragen bis 3 km und darf sie annehmen. PLZ-Gleichheit kann die km-Prüfung vollständig aushebeln (ländliche PLZ = über 10 km).
- Der UI-Text „…oder im Suchradius **des Anbieters**" (`mobile/.../booking/[id].tsx:613`, Web analog) beschreibt die Implementierung falsch: Es ist der Radius des **Suchenden**.

C. Direktanfrage umgeht jeden Radius HOCH

- `createBooking` prüft nur Zeitfenster und Typ (`booking/new.tsx:198–209`; `isBookingCoveredByAvailabilities` ohne Distanz, `availabilities/queries.ts:296–321`).
- Kein Client- und kein Server-Check gegen #2 oder #3. Ein Helfer-Detail-Link (`/helper/[id]` → `/booking/new?helperId=…`) erlaubt Buchungen außerhalb des Such- und Reiseradius.

D. Gefahren-Radar vs. Gefahren-Detail HOCH

- Der Radar-RPC liefert Gefahren, deren **Zentrum** im Radar-Radius liegt (bis 3 km), unabhängig vom Warnradius – SECURITY DEFINER umgeht die RLS (`044:38–50`).
- Das Detail lädt direkt aus `hazards` (`hazards/queries.ts:58–64`) und unterliegt der RLS „innerhalb des Warnradius“ (0,5/1,5 km, `006:67–72`).
- Ergebnis: Eine Gefahr bei 2,5 km erscheint in der Liste, das Detail ist aber nicht lesbar (PostgREST-Fehler); Fotos (`014:81–91`) und Sichtungen (`007:42–51`) ebenfalls nicht.
- Zusätzlich hängt das Detail am **gespeicherten** Profilstandort, der Radar am **Live-GPS**.

E. Vermisst-Radar: gleiche Doppellogik, andere Richtung MITTEL

- Sichtbarkeit wird vom **Melder-Radius** bestimmt (3 km), nicht von einem Radar-Radius. Es gibt keinen Radius-Regler im Vermisst-Radar – anders als beim Gefahren-Radar.
- DB-Default 2 km, beide Clients setzen 3 km: Ein Insert ohne expliziten Radius (z. B. Admin/Skript) wäre inkonsistent.
- Auch hier: Live-GPS im Radar vs. gespeicherter Profilstandort in der RLS.

F. Zwei verschiedene Entfernungen für denselben Helfer MITTEL

- Liste: Live-GPS → Helfer-Profilstandort (`find_nearby_helpers`).
- Detail: `get_helper_detail` rechnet vom **gespeicherten** `profiles.location` (`20261025120000:101–127`) – andere Zahl oder „–“.
- Der als „ca.“ markierte Detailwert kann daher von der Listenangabe abweichen, obwohl beide dieselbe Einheit und Formatierung nutzen.

G. Radius-Obergrenzen laufen kreuz und quer MITTEL

- Helfer-Reisebereitschaft 50 km > Explore 10 km > Profil-Kiez 3 km = Gefahren-Radar 3 km = Vermisst 3 km > Warnradius 1,5 km > Tracking 10 km (in Metern).
- Kein gemeinsames Konstantenmodul: KIEZ-Optionen doppelt gepflegt (`shared/constants/app.ts` und `supabase/profile/queries.ts`), RPC-Grenzen je Migration neu, UI-RADII je Screen. Drift ist vorprogrammiert.
- Beispiele: Doku sagt 500 m–5 km (`pfotennetz_gesamte_plattform.md:453`); das E2E-Fixture nutzt 5 km (`web/e2e/smoke.spec.ts:152`), was der DB-Check ablehnt.

H. Defaults widersprechen sich NIEDRIG

- Helper-Suche: RPC 2 km vs. UI 3 km.
- Gefahren-Radar: RPC 5 km vs. UI/Default 1,5 km.
- Warnradius: DB 0,5 vs. Shared-Schema 1,5 vs. UI 0,5/1,5.
- Vermisst: DB 2 km vs. UI 3 km.
- Tracking: Migration `20261024150000` Default 500 m → `20261024160000` Default 1 000 m (Rows angehoben); Web-Geofence-Stub hat noch 500 als Default (`geofence.web.ts:6`, aktuell ungenutzt).

I. Semantik des Warnradius MITTEL

- „Warnradius“ klingt wie „wer wird gewarnt“. Er steuert aber nur die Lesbarkeit (RLS) und hat **keine** Notification-Wirkung.
- Copy wie „Menschen im Warnradius informiert“ (`hazard/success.tsx:19`) und die Doku „Push im 1,5 km-Radius“ (`pfotennetz_gesamte_plattform.md:211`) sind nicht implementiert.
- Der Radar listet nach Zentrumsdistanz, nicht nach Überlappung des Warnkreises: Eine Gefahr mit 1,5 km Warnradius bei 2,9 km Distanz liegt außerhalb ihres eigenen Radius, wird aber angezeigt.

J. Kleinere Beobachtungen NIEDRIG

- Explore-Karte beschriftet „ca. ±{radiusKm} km“ (`explore.tsx:147`) – „±“ ist bei einem Suchradius irreführend.
- Listen-Entfernungen werden ohne „ca.“ gezeigt, obwohl serverseitig auf 2 Nachkommastellen (10 m) gerundet wird; das Detail schreibt „ca.“ und rundet Koordinaten auf ca. 110 m.
- Beim Annehmen offener Anfragen wird die **Verfügbarkeit** (Tag/Zeit/Typ/Tierart) ebenfalls nicht geprüft – nur Distanz/PLZ. Das verstärkt Punkt B.

4. Entscheidungen (8. November 2026)

Frage	Entscheidung
Profil-Radius	Reichweite + Suchstart: offene Anfragen, Umgebungs-Statistik und Startradius der Helfersuche.
PLZ-Regel	Gleichwertiger zweiter Weg neben der Distanz (kein reiner Fallback).
Helfer-Slot	Für die Reisebereitschaft zählt nur die Betreuungsart, nicht Tag oder Uhrzeit.
Warnradius	Warnkreis-Überlappung für Liste, Detail, Fotos und Sichtungen.
Direktbuchung	Der Helfer-Radius wird serverseitig erzwungen.

5. Umsetzungsstand

Befund	Status	Umsetzung
A	erledigt	Zentrale Konstanten, Web-Label „Nachbarschafts-Radius“, Explore startet mit dem Profil-Radius (0,5–3 km als Start, bis 10 km erweiterbar).
B	erledigt	Offene Anfragen: Seeker-Radius bzw. gleiche PLZ **und** Reisebereitschaft des Helfers für die Betreuungsart.
C	erledigt	Trigger beim Anlegen einer Direktanfrage plus Client-Vorprüfung; ein Detail-Link umgeht den Radius nicht mehr.
D	erledigt	Warnkreis-Überlappung im Radar; Detail, Fotos und Sichtungen nutzen dieselbe Regel (max. Radar 3 km + Warnradius).
E	teilweise	DB-Default der Vermisst-Meldung auf 3 km angeglichen. Der Vermisst-Radar bleibt bewusst ohne eigenen Regler; der Meldungsradius bestimmt die Sichtbarkeit.
F	offen	Explore-Distanz nutzt Live-GPS, das Helfer-Detail den Profilstandort. Bewusste Entscheidung; die Zahlen können leicht abweichen.
G	erledigt	Alle Radien in `packages/shared/src/constants/geo.ts`; die fachlich unterschiedlichen Obergrenzen bleiben erhalten.
H	erledigt	UI-, Shared- und SQL-Defaults angeglichen (1,5 km Suche, 1,5 km Radar, 0,5 km Warnradius, 3 km Vermisst).
I	teilweise	Copy korrigiert: keine Prüf- oder Push-Versprechen mehr. Push-Benachrichtigungen für Gefahren bleiben offen.
J	erledigt	„Radius X km“ statt „±“, einheitliche Formatierung (500 m / 1,5 km).

Migrationen: `20261108120000_nearby_helper_count_admins`, `20261108130000_booking_helper_radius`, `20261108140000_hazard_warning_overlap`, `20261108150000_radius_defaults` – noch auf die Zielumgebung anzuwenden.

Prüfungen: Typecheck 7/7 Projekte, Lint grün (eine bestehende Bild-Warnung im Web), Tests: 58 shared, 123 supabase, 96 mobile.
Abschnitt 3 beschreibt den Ausgangsbefund vom 4. November 2026. Abschnitt 4 und 5 dokumentieren die Entscheidungen und die Umsetzung vom 8. November 2026. Versionen/Releases wurden nicht angehoben; die Web-Rebuild-Regel aus `AGENTS.md` ist daher noch nicht ausgelöst.